Recursividade

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Iteratividade x Recursividade

- Fatorial iterativo:
  - **n! = 1 se n ≤ 0**
  - **n.(n-1).(n-2). . . . 3.2.1 se n > 0**
- Fatorial recursivo:
  - n! = 1 se n ≤ 0
  - n.(n-1)! se n > 0
- Definição recursiva: uso da função para definir a própria função -> Ou seja, se f é a função fatorial, a definição acima fica:
  - f(n) = 1 se n ≤ 0
  - n.f(n-1) se n > 0

5!=5.4!=5.4.3!=5.4.3.2!=5.4.3.2.1!=5.4.3.2.1.0!=5.4.3.2.1.1 = 120

Iteratividade x Recursividade

- A definição recursiva faz sentido pois tem-se um caso em que a recorrência termina: quando n == 0 -> não há mais chamadas.

- Muitas outras funções admitem definições recorrentes deste tipo, ou seja, usa-se a função para definir a si própria
  - **é NECESSÁRIA a existência de um caso particular não recorrente e a GARANTIA de que a recorrência levará a esse caso particular.**

Recursão em Python

- Fatorial:

```
def fatorial(n):
    if n == 0: return 1
    print("Chamando fatorial de ", n - 1)
    return n * fatorial(n - 1)

f = fatorial(5)
print("Fatorial de 5 = ", f)
```

Números Harmônicos

- Função harmônica iterativa : **H(n) = 1 + 1/2 + 1/3 + 1/4 + . . . + 1/n (n >= 1)**
- Função harmônica recursiva:
  - H(n) = 1 se n ≤ 1
  - 1/n + H(n-1) se n > 1
- Exemplo:
  - H(4)=1/4+H(3)=1/4+1/3+H(2)=1/4+1/3+1/2+H(1)=1/4+1/3+1/2+1
- Implementação em Python:

```
def H(n):
    if n <= 1: return 1
    return 1 / n + H(n - 1)
```

Soma dos elementos de uma lista

- Considere uma lista de números, por exemplo [23, 45, 87,12] dos quais queremos a soma.
  - Abordagem iterativa: percorrer a lista, acumulando a soma dos elementos.
  - Abordagem recursiva:
    - Soma da lista = 0 (se a lista está vazia)
    - primeiro elemento + Soma do resto da lista (se a lista não está vazia)

```
def soma_da_lista(lista):
    if lista == []: return 0
    return lista[0] + soma_da_lista(lista[1:])
```

Soma dos elementos de uma lista

Exercício (implementar versões iterativa e recursiva):

1. Função Max_Lista(L) que devolve o máximo dos elementos de uma lista L.
2. Idem Min_Lista(L) para o mínimo.
3. Função Inverte_Lista(L) que devolve a lista L invertida.
4. Função Potencia(x, n) que recebe x ≠ 0 float e n > 0 inteiro e devolve x^n
5. Função Soma_Seq(x, n) que recebe x float e n ≥ 0 inteiro e devolve o valor da soma:
    a. x + x^2/2 + x^3/3 + ... x^n/n
6. Função Soma_Seq(x, eps) que recebe x float e eps float (eps - valor pequeno – ex: 10^-5; x é um valor entre 0 e 1) e devolve o valor da soma acima (exercício 5) até encontrar um termo que seja menor que eps
    a. Mais detalhes: agora, você não sabe mais *n* de antemão. Em vez disso, você deseja somar os termos enquanto eles forem maiores que um pequeno valor eps, ou seja, parar quando o termo da série for menor que eps
7. Função Busca(L, x) que recebe uma lista L e um valor x e procura x em L. Devolve o primeiro k tal que L[k] = x ou -1 se não encontrar. Está resolvido à frente, mas tente resolvê-lo sem ver a solução.
8. Função Binomial(n, k) que devolve o número binomial de n e k. A definição recursiva do número binomial de n e k é:
    a. 0 se se n = 0 e k > 0
    b. 1 se se n ≥ 0 e k = 0
    c. Binomial(n-1, k) + Binomial(n-1, k-1) se n e k > 0

Busca dos elementos de uma lista

1. Busca elemento máximo da lista (recursivo)

```
def maior(x, y):
    if x > y: x ? y

def Max_Lista(L):
    if len(L) == 1: return L[0]
    return maior(L[0], Max_Lista(L[1:]))
```

Observações sobre implementações recursivas

- Nos casos acima, não parece existir nenhuma vantagem da solução recursiva para a solução iterativa.
  - Solução recursiva: até pior em termos de gasto de memória e de tempo, pois cada chamada guarda o contexto da chamada anterior, até que ocorra o caso de parada.
- Vantagem: a estrutura da função recursiva fica análoga à sua definição.
- Recursão: procura expressar a solução de um problema em termos de soluções mais simples (ou menores) do mesmo problema, até chegar ao ponto de resolução trivial
  - Critério de parada.
- Entretanto em alguns casos a recursão permite escrever soluções que, de outra forma, seriam muito difíceis de implementar.

Observações sobre implementações recursivas

- Soluções recursivas sempre possuem as seguintes características:
  - O algoritmo deve possuir um caso base (critério de parada), ou um caso em que não seja necessária a chamada de si mesmo.
  - O algoritmo deve mutacionar-se de maneira a se aproximar do caso base.
  - O algoritmo deve ter uma chamada a si mesmo (direta ou indiretamente) -> dentre os comandos (inclusive chamadas de funções que o compõe) deve haver uma chamada de si mesmo.

Sequência de Fibonacci

- Proposta pelo matemático italiano Leonardo Fibonacci: cada elemento (com exceção dos dois primeiros que são 0 e 1), é igual à soma dos dois anteriores.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, . . .

- Assim:
  - f(0) = 0
  - f(1) = 1
  - f(n) = f(n – 1) + f(n – 2)

Sequência de Fibonacci

Solução iterativa

```
def Fibonacci(n):
    if n == 0: return 0   # casos
    if n == 1: return 1   # particulares
    anterior, atual = 0, 1
    # repetir até o n-ésimo inclusive
    for k in range(2, n + 1, 1):
        anterior, atual = atual, anterior + atual
    return atual
```

Solução recursiva

```
def Fibonacci(n):
    if n == 0: return 0   # casos
    if n == 1: return 1   # particulares
    return Fibonacci(n - 1) + Fibonacci (n - 2)
```

Mais elegante, porém menos eficiente.

- Quantas chamadas são necessárias, para se calcular Fibonacci(n) n ≥ 0?

Sequência de Fibonacci

Solução iterativa

```
def Fibonacci(n):
    if n == 0: return 0   # casos
    if n == 1: return 1   # particulares
    anterior, atual = 0, 1
    # repetir até o n-ésimo inclusive
    for k in range(2, n + 1, 1):
        anterior, atual = atual, anterior + atual
    return atual
```

Solução recursiva

```
def Fibonacci(n):
    if n == 0: return 0   # casos
    if n == 1: return 1   # particulares
    return Fibonacci(n - 1) + Fibonacci (n - 2)
```

Mais elegante, porém menos eficiente.

- Quantas chamadas são necessárias, para se calcular Fibonacci(n) n ≥ 0?

Para Fibonacci(3) - 5 chamadas

Máximo Divisor Comum

- Problema: Algoritmo de Euclides
  - MDC(x, 0) = x
  - MDC(x, y) = MDC(y, x % y)

Quociente		1	1	2
dividendo/divisor	30	18	12	6
Resto	12	6	0	

MDC(30, 18) = MDC(18, 12) = MDC(12, 6) = 6

Implementação iterativa

```
def mdc(n, m):
    dividendo, divisor, resto = n, m, n % m
    while resto != 0:
            dividendo, divisor, resto = divisor,
            resto,  divisor % resto
    return divisor
```

Implementação recursiva

```
def mdc(n, m):
    if m == 0: return n
    return mdc(m, n % m)
```

Busca Sequencial

- Implementação iterativa

```
# Procura x na lista a de n elementos
# Devolve o índice do primeiro encontrado ou -1 se não encontrar
def busca(a, n, x):
    for k in range(n):
        if a[k] == x: return k
    return -1
```

Busca Sequencial

- Implementações recursivas

```
# Procura x na lista a de n elementos
# Devolve o índice do primeiro encontrado ou -1 se não encontrar
def busca(a, n, x, k):
    if k == n: return -1
    if a[k] == x: return k
    return busca(a, n, x, k+1)

# exemplo de chamadas da função busca

v = [1, 2, 3, 4]
print(busca(v, 4, 3, 0))
print(busca(v, 4, 5, 0))
```

Busca Sequencial

- Implementações recursivas

```
# Procura x na lista a de n elementos
# Devolve o índice do último elemento igual a x ou -1 se não encontrar
def busca(a, n, x):
    if n == 0: return -1
    if a[n - 1] == x: return n - 1
    return busca(a, n - 1, x)

# exemplo de chamadas da função busca

v = [1, 2, 3, 4]
print(busca(v, 4, 3))
print(busca(v, 4, 5))
```

Busca Sequencial

- Implementações recursivas

```
# Procura x na lista a
# Devolve True ou False - usando sub-listas
def busca(a, x):
    if len(a) == 0: return False
    if a[0] == x: return True
    return busca(a[1:], x)
```

Exercício 9: Implemente a busca acima varrendo do final para o começo da lista.

Busca Binária

- Problema:
  - Compare com o elemento do meio da lista.
    - Se == : achou.
  - Se > : faça a Busca na parte inferior da tabela.
  - Se < : a Busca na parte superior da tabela.
- Implementação recursiva:

```
def Busca_Binaria(L, x, inicio, fim):
   if inicio > fim: return -1 # tabela vazia
   meio = (inicio + fim) // 2
   if x == L[meio]: return meio
   if x > L[meio]: return Busca_Binaria(L, x, meio + 1, fim)
   return Busca_Binaria(L, x, inicio, meio - 1)
```

Exercício 10: Reescreva a função acima apenas com o parâmetros L e x (def Busca_Binaria(L, x)). Use o tamanho de L e sub-listas.

Potência com expoente inteiro

- Seja f(x, n) = x^n (n ≥ 0 inteiro).
- Observe que f(x, n) = 1 se n = 0 e x * f(x, n - 1) se n > 0

Exercício 11: Escreva a função acima nas versões iterativa e recursiva.

- A solução recursiva acima não apresenta vantagens, pois continuamos fazendo n multiplicações. Será possível diminuir a quantidade de multiplicações?

Potência com expoente inteiro

- Veja que:
  - x^4 = x^2*x^2 e x^2 = x*x (x^4 com 2 multiplicações)
  - x^5 = x^4*x , x^4 = x^2*x^2 e x^2 = x*x (x^5 com 3 multiplicações)
  - x^10 = x^5*x^5 , x^5 = x^4*x , x^4 = x^2*x^2 e x^2 = x*x (x^10 com 4 multiplicações)

$$x^n = \begin{cases} 1 \text{ se } n = 0 \\ (x^h)^2 \text{ se } n > 1 \text{ e par} \\ (x^h)^2 . x \text{ se } n > 1 \text{ e ímpar} \\ \text{Onde h é o quociente de n/2 truncado.} \end{cases}$$

Exercícios

12. Faça uma função pot(x, n) recursiva usando a definição acima. Verifique na sua solução quantas chamadas à função pot são feitas. O objetivo é que a função faça o mínimo de chamadas possível.
13. Quantas multiplicações são feitas para se calcular x^n no algoritmo acima ?
14. Refaça os 2 exercícios anteriores expandindo a definição para *n* negativo, lembrando que:
    a. x^n = 1 / x^-n se n < 0. Se n < 0 e x = 0 a função não está definida.
15. Dado *n* e uma sequência com *n* números, imprimir a sequência na ordem inversa a que foi lida. Fazer isso sem usar listas. Sugestão: faça uma função recursiva imprime, que lê um número, chama a si própria se não chegou ao fim da sequência e imprime o número lido.

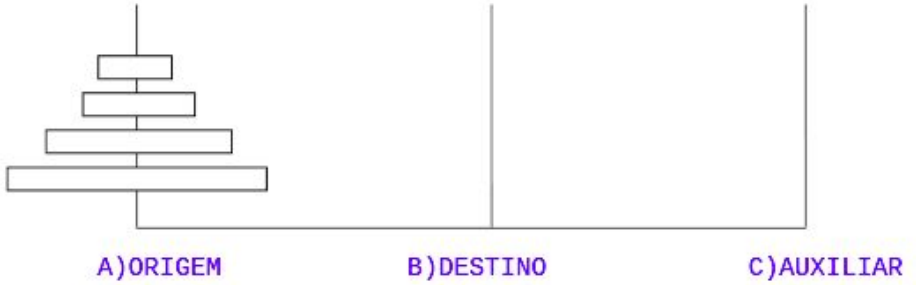
Torre de Hanói

- Todos os exemplos anteriores:
  - versão recursiva mais elegante
  - não é mais eficiente que a versão iterativa
- Em alguns casos, no entanto, pensar na solução de forma recursiva, facilita a resolução do problema.
- O jogo Torre de Hanói, foi inventado pelo matemático francês Édouard Lucas (1842-1891)
  - é um exemplo clássico de problema que tem uma solução facilitada na forma recursiva.
- O problema: mover *n* discos empilhados (os menores sobre os maiores), de uma haste de origem para uma haste de destino na mesma ordem respeitando as seguintes regras:

Torre de Hanói

Regras:

- Apenas um disco pode ser movido por vez
- Não colocar um disco maior sobre um menor
- Pode-se utilizar uma haste auxiliar



Por exemplo, uma solução para 2 discos seria:

```
move disco   1 da torre     ORIGEM para a torre   AUXILIAR
move disco   2 da torre     ORIGEM para a torre    DESTINO
move disco   1 da torre   AUXILIAR para a torre    DESTINO
```

Correspondente aos movimentos abaixo:

```
| 1          |            |          |          |
| 2          | 2      1   |      2 1 |      1   |
| A  B  C    | A  B  C    | A  B  C  | A  B  C  |
```

Torre de Hanói

- Implementação em Python

```python
def Movimente(k, origem, destino):
   print("mover disco ", k, " da torre ", origem, " para a torre ", destino)

def Hanoi(n, torreA, torreB, torreAux):
   if n == 1:
       # mover disco 1 da torreA para a torreB
       Movimente(1, torreA, torreB)
   else:
       # n - 1 discos da torreA para torreAux com torreB auxiliar
       Hanoi(n - 1, torreA, torreAux, torreB)
       # mover disco n da torreA para torreB
       Movimente(n, torreA, torreB)
       # n - 1 discos da torreAux para a torreB com torreA auxiliar
       Hanoi(n - 1, torreAux, torreB, torreA)
```

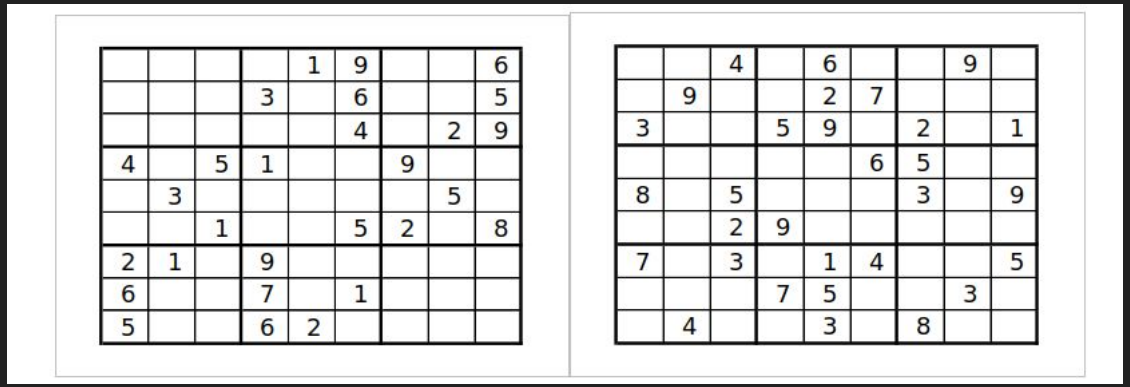
Torre de Hanói

- Exemplo com 3 discos (Hanoi (3, A, B, C))

```
Hanoi(3, A, B, C)
├── Hanoi(2, A, C, B)
│   ├── Hanoi(1, A, B, C)
│   │   └── Movimente(1, A, B) - menor disco (1) em B
│   ├── Movimente(2, A, C) - disco do meio (2) em C
│   └── Hanoi(1, B, C, A)
│       └── Movimente(1, B, C) - menor disco (1) em C (acima de (2))
├── Movimente(3, A, B) - maior disco (3) em B
└── Hanoi(2, C, B, A)
    ├── Hanoi(1, C, A, B)
    │   └── Movimente(1, C, A) - menor disco (1) em A
    ├── Movimente(2, C, B) - disco do meio (2) em B (acima de (3))
    └── Hanoi(1, A, B, C)
        └── Movimente(1, A, B) - menor disco (1) em B (acima de (2))
```

Sudoku

- O jogo Sudoku consiste em preencher os espaços vazios de uma matriz 9x9 com os algarismos de 1 a 9 de tal maneira que em cada linha, cada coluna e cada quadrado interno de 3x3 contenha todos os algarismos sem repetição.
- O jogo possui graduações de dificuldade. Normalmente é publicado como Muito Fácil, Fácil, Médio, Difícil e Muito Difícil. Essa graduação está associada à quantidade de possibilidades para se chegar à solução. Quanto mais possibilidades, mais difícil.



				1	9			6
			3		6			5
					4		2	9
4		5	1			9		
	3						5	
		1			5	2		8
2	1		9					
6			7		1			
5			6	2				

		4		6			9	
	9			2	7			
3			5	9		2		1
					6	5		
8		5				3		9
		2	9					
7		3		1	4			5
			7	5			3	
	4			3		8		

Sudoku

- Para cada posição vazia, pode haver mais de um candidato:
  - Solução inicial: colocar os candidatos em cada uma das posições até encontrar uma combinação de candidatos que preencha toda a matriz. Pode haver mais de uma solução ou eventualmente nenhuma.
- Solução convencional: experimentar o preenchimento de cada posição por tentativa e erro. O mesmo princípio pode ser usado num algoritmo.
- Preencher a matriz S:
  - Procure uma casa livre – seja S[lin][col] essa posição:
    - Se não há posição livre já temos uma solução
  - Para cada um dos candidatos a essa posição:
    - Faça S[lin][col] = candidato
    - Preencher a matriz S (agora com mais uma posição preenchida)
  - Faça S[lin][col] = 0 (não há candidato que atenda a essa posição – retorne dessa chamada)

Veja que estamos invocando o preenchimento dentro do preenchimento, mas com pelo menos uma posição a mais já preenchida.



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr